



BENCHMARK REPORT

Native Movement Response and Validation

Matched-state response, dot-core isolation, and camera-to-display evidence.

2026 EDITION / IAIN BENNETT / MIT LICENSE

VirtualGlove began with a deceptively simple question: **when the player moves their hand, does the Robo-Glove move promptly, accurately, and for the reason we think it does?**

Answering that question took more than pointing a camera at the screen. A delay can begin in the camera, recognition, network, console receiver, emulator, or display. A smooth-looking result can hide stale frames. A fast software trace can miss the delay a player actually feels. We therefore measured the system in layers, changed one layer at a time, and kept live play as the final judge.

This guide tells that investigation as an engineering story. It explains what we tested, what failed, what was promoted into VirtualGlove, and what the numbers do-and do not-prove. The final sections retain the supported procedures needed to repeat the useful measurements.

The result in one minute

VirtualGlove now sends the newest valid MediaPipe hand position directly to the console. It does not predict where the hand will go, replay queued movement, or smooth normal motion through a delayed tail.

The work established five important results:

1. **Native Super Glove Ball input really is continuous X/Y input.** The custom Nestopia core responds to small coordinate changes, not merely left/right/up/ down extremes.
2. **The native and FCEUmm paths react on the same visible emulated frame for the exact Super Glove Ball ROM tested.** Both first changed the picture on frame 3, about 50 ms into a 60 Hz emulation run.
3. **A same-ROM comparison found a real bug.** Native Y wrapped incorrectly at an endpoint even though earlier checksum tests looked healthy. The corrected packet now places the Robo-Glove at bottom, centre, and top as intended.
4. **Fresh measured input beat speculative complexity.** Optical flow, extra smoothing, parallel trackers, larger images, and several accelerated inference paths either added age, weakened recognition, or failed to improve the whole path.
5. **The remaining long tail is mostly reacquisition.** Normal landmark tracking is faster than searching for a hand again. Good lighting and keeping the hand in view matter more than adding another movement filter.

These results support the production design. They do **not** turn a headless emulator measurement into a universal camera-to-display latency claim.

The path we had to measure

There are two gameplay routes after the shared camera and recognition stages:

- **Standard games through FCEUmm** receive familiar digital directions and NES buttons. Hand position is classified against the player's saved centre box.
- **Super Glove Ball through Nestopia (VirtualGlove)** receives absolute native X/Y coordinates plus the Power Glove packet state expected by the game.

Both routes begin with the same physical hand, camera frame, MediaPipe result, player calibration, and authenticated Controller-to-console delivery. They only diverge at the console output and emulator.

That distinction matters. FCEUmm can say "hold Right." Native input can say "place the Robo-Glove here." Similar visible response times do not make those controls equivalent.

Four layers of evidence

We used four complementary tests because no single benchmark sees everything:

Layer	What it isolates	What it cannot prove
Same-ROM headless benchmark	Emulator and input-path response from a controlled state	Camera, network, display, or human-perceived delay
Calibration dot core	Native centre, reach, clamping, stability, loss, and recovery without game logic	A commercial game's interpretation of the packet
Controller and receiver telemetry	Capture, inference, send, validation, and publication stages	Physical display response by itself
High-frame-rate hand-and-screen video	The complete experience from visible hand motion to visible game motion	The internal stage responsible without matching telemetry

The layers are intentionally separate. Percentiles from different clocks are not added together, and local send success is never presented as proof that the emulator consumed an input.

Establishing a trustworthy emulator baseline

The first job was to remove the camera and network from the question. The benchmark boots an exact ROM into play, saves one emulator state, and restores that state for every comparison. One run remains neutral; the other changes only the candidate input. The first video frame with a different checksum is the visible response.

Release is measured the same way. After eight frames of held input, continuing the hold is the baseline and returning to neutral is the only change.

Reproducible inputs

Lane	Core	Exact game image
Native coordinates	Nestopia PowerGlove built from pinned Nestopia revision 5a1cd378cb46ca9ccc2dd6f8b2b6a79ab986052e plus the repository patch	Super Glove Ball (USA) , SHA-256 ad60ef1b62cd1b3bc02a9320376067347a8ab2ebbe46e1616693d8379c9d9a7b
Standard joystick	Stock FCEUmm revision 236ccdffc911e84c60fea6b9d0699c2d440a8de14	The same exact Super Glove Ball (USA) image
Optional D-pad reference	The same stock FCEUmm revision	Gun Smoke (USA) , SHA-256 4ad9629a2bacc158a7f50975869c7dfe533567ae399a5bdc5df2240286df259f

ROMs stay outside the project. Results are tied to digests so a similarly named image cannot silently change the experiment.

What the emulator test found

Three complete executions produced byte-identical reports.

Lane	Directions	First visible activation	First visible release	Input evidence
Super Glove Ball / lr-nestopia-powerglove	Left, right, up, down	Frame 3, about 50.0 ms at 60 Hz	Frame 3, about 50.0 ms at 60 Hz	Native state published before the frame; packet trace confirms per-frame consumption
The same ROM / stock FCEUmm	Left, right, up, down	Frame 3, about 50.0 ms at 60 Hz	Frame 3, about 50.0 ms at 60 Hz	Standard joystick callback polled; no native packet input
Gun Smoke / stock FCEUmm	Left, right, up, down	Frame 2, about 33.3 ms at 60 Hz	Frame 2, about 33.3 ms at 60 Hz	Standard joystick callback polled

The native positive-X sweep also changed the picture on frame 3 at every tested magnitude: [1024](#), [2048](#), [4096](#), [8192](#), [16384](#), and [32767](#). The smallest step is about 3.1% of the positive signed range. That is the evidence that the native path preserves small continuous movement rather than disguising a D-pad behind native terminology.

The comparison also caught the Y-axis endpoint error. The corrected packet trace returns [\\$80](#), [\\$00](#), and [\\$7F](#) for Y minimum, centre, and maximum, and the game places the Robo-Glove at bottom, centre, and top. This was exactly the kind of fault the layered method was meant to expose: transport was functioning, but the visible meaning was wrong.

Making the camera path feel attached to the hand

Once the emulator path was understood, attention moved upstream. The goal was not the smallest isolated inference number. It was the freshest dependable hand position delivered during real play.

First rule: never build a frame queue

Camera capture continuously replaces one latest-frame slot. If recognition is busy, older unprocessed frames are discarded. This sacrifices redundant frames to prevent a growing delay—the right trade for direct hand control.

The same principle continues after recognition. The Controller sends a fresh state immediately. Preview annotation and JPEG encoding run separately and may discard superseded previews rather than delaying gameplay.

The optical-flow detour

Optical flow initially looked attractive: estimate hand motion between slower MediaPipe results and make the glove appear to follow at camera rate. A synthetic UNO Q benchmark even looked encouraging. A revised implementation reduced foreground processing p95 from roughly 79 ms to 18 ms and recognition-source age from roughly 266 ms to 130 ms.

The live hold test told a different story. Only 17 of 128 still-hold samples were detected, compared with 177 of 177 for the original MediaPipe path. The correction work replayed intervening frames and then submitted the next recognition job from an already-aged source. It made a local benchmark faster while making the actual control path less dependable.

We rejected the experiment. Optical flow was disabled, then removed in the 0.4.2 cleanup. Git history remains the archive. This failure established an important rule for the project: **an optimization must improve the complete control experience, not merely its own stopwatch.**

Finding the useful MediaPipe settings

Repeatable guided clips let different recognition settings see the same pixels. The early 52-second session covered neutral, movement, buttons, rolls, closed hand, depth gestures, tracking loss, and near/far poses.

The broad conclusions were more valuable than any single early number:

- 512×384 did not improve p95 enough and increased neutral false activations.
- MediaPipe Tasks Video found some difficult poses but approximately doubled median inference time.
- Larger search areas stopped helping beyond the selected 2.25 scale.
- Four inference threads gave the best UNO Q reacquisition result without a repeatable thermal or scheduling penalty.
- The complete 640×480 MediaPipe Hands graph preserved the best overall balance.

The selected production baseline became 640×480, four inference threads, tracking confidence 0.35, and a 2.25 next-frame search region.

Improving fast sweeps without chasing noise

A 402-frame sweep clip compared the ordinary next-frame search with a gentle direction-aware translation. Detection continuity improved from 95.52% to 96.52%, and counted reacquisitions fell from six to four. The selected settings were gain 0.275, activation speed 0.50, and maximum translated offset 0.04.

Frame-level review then corrected an apparent failure: the longest missing run occurred during the scripted instruction to remove the hand. It was expected tracking loss, not a bad sweep. The actual fast-X/Y cue detected 91 of 92 frames. This is why aggregate percentages were always checked against the cue timeline.

A matched lighting comparison reinforced practical setup advice:

Lighting	Overall detection	Fast-sweep detection	Missing frames	Reacquisitions
Strong window backlight	98.73%	98.82%	3	2
Reduced backlight	99.57%	98.84%	1	1

The result does not justify universal threshold changes, but it does explain why front lighting and a less dominant bright window make recovery more reliable.

Updating the recognition runtime

The same private 736-frame sweep clip compared MediaPipe 0.10.18 and 0.10.35 with identical settings:

Runtime	Inference p50 / p95	Continuity	Landmark continuation p95	Reacquisition p95
0.10.18	36.12 / 60.18 ms	97.96%	51.10 ms	149.48 ms
0.10.35	34.03 / 46.60 ms	97.96%	44.29 ms	120.16 ms

MediaPipe 0.10.35 kept continuity while lowering every timing column. A live 3,408-sample trace had no drops and felt quicker, so it became the sole shipped recognition runtime.

Choosing the production camera path

Camera research tested OpenCV, direct V4L2, thread isolation, process isolation, buffer counts, frame-rate requests, exposure modes, and a Kiyo-specific HDR-off experiment. These tests produced useful instrumentation, but the final choice came from matched live Super Glove Ball play.

Direct V4L2 supplied valuable driver timestamps and showed that the Kiyo Pro could deliver 640×480 MJPEG near 30 fps reliably. Process isolation kept camera capture moving through a long inference call, which made it useful as an engineering comparison. Yet the player found both direct-V4L2 lanes less responsive than OpenCV in the actual game.

The selected production path is therefore:

- OpenCV capture with thread isolation;
- the complete 640×480 MediaPipe graph;
- four inference threads;
- a one-buffer request and newest-frame replacement;
- a 30-fps-first automatic camera policy with safe fallback;
- automatic exposure by default.

In the final OpenCV live session, 49.66 seconds of play produced 22.29 coordinate results per second and 97.3% detection, including an intentional departure. Camera-read completion to send was 61.2/84.7 ms p50/p95, and inference was 40.2/59.6 ms. OpenCV cannot expose sensor and driver residency the way direct V4L2 can, so those numbers are not directly interchangeable with driver-timed results. Live feel and end-to-end continuity decided the production choice.

UNO Q Kiyo Pro capture comparison - September 6, 2026

The historical Kiyo experiment explains why "request 60 fps" is not the same as "receive fresh useful control at 60 fps." With HDR disabled and two buffers, 640×480 capture reached about 59.7 fps in an isolated capture-only test; 720p decoded more slowly and had longer tails. Under recognition load, however, 30-fps-first play felt smoother and more attached to the hand.

The experiment did not isolate HDR's contribution and did not include inference or physical hand-to-screen timing. It led to reversible camera controls and better diagnostics, not a universal Kiyo preset. Automatic exposure remains the portable default. On the project's Kiyo Pro, manual exposure 78 and gain 96 slightly improved continuity in one matched live comparison without changing latency, but those values belong to that camera and room.

What remained slow-and why we stopped tuning

Sustained tracing separated normal landmark continuation from palm detection. The rare long events came from [palm_detection_no_valid_hand](#), not from normal tracking. In one five-minute stress lane, a 260.53 ms palm-search call was followed by a 190.17 ms camera dequeue interval and a 327.19 ms coordinate-age maximum. The camera recovered immediately.

Further experiments tried three inference threads, capture in a separate process, wider search regions, alternate tracking confidence, one-frame region grace, and parallel or staggered trackers. Some improved an isolated boundary; none produced a repeatable live benefit without losing results, adding age, or weakening continuity.

That was the stopping point. The evidence identified palm reacquisition as the remaining tail, but did not support hiding it with prediction or another queue. VirtualGlove instead keeps normal movement direct, briefly holds the last native X/Y through a small gap, and neutralizes safely after stale input.

What the evidence proves

The tests establish that:

- the custom core consumes coherent native state and responds continuously;
- the exact tested Super Glove Ball ROM reacts on frame 3 in both native and FCEUmm lanes;
- the corrected native endpoints match visible game placement;
- current recognition settings preserve strong continuity on the retained clips;
- the signed transport and receiver add little processing time compared with camera and recognition;
- the deployed newest-frame design avoids a deliberate software backlog;
- OpenCV with thread isolation is the best live-tested camera path on the UNO Q.

They do not establish one universal end-to-end latency number. Camera exposure, USB/driver buffering, recognition path, network, emulator timing, display buffering, and panel response all vary. A physical claim still requires the hand and screen in the same verified high-frame-rate recording.

Direct-output dot test

Use **VirtualGlove Calibration Test** when Super Glove Ball makes X/Y behaviour hard to judge. The installer can add this ROM-free item to RetroPie's **Ports** list. Its [lr-powerglove-dot](#) core uses the normal camera, recognition, calibration, signed delivery, receiver, and native-state publication. Only the game interpretation is replaced by a dot.

1. Select the player, confirm calibration, and start Controller delivery.
2. Launch **Ports -> VirtualGlove Calibration Test**.
3. Move through centre and the saved reach. The yellow dot should follow the hand. A green marker means the sample is valid and fresh; a red X means no valid input.
4. Hold still for five seconds, make three horizontal and three vertical moves with one-second holds, then leave and re-enter the camera view.
5. Exit normally so traces finalize. Repeat in Super Glove Ball without changing player or camera settings.

This isolates centre, reach, edge clamping, stationary stability, loss, and recovery. It does not score gestures or prove a particular ROM's packet logic.

Collect a live status baseline

Use the read-only collector before changing responsiveness settings. Keep camera position, lighting, calibration, game state, and recognition settings fixed. Prepare the intended mode on Dashboard first; the collector does not start the camera or controller output.

```
SH
python3 scripts/measure-vision-status.py \
  --status-url http://CONTROLLER.local:8088/status \
  --phase neutral --seconds 30 --output /tmp/virtualglove-neutral.json
```

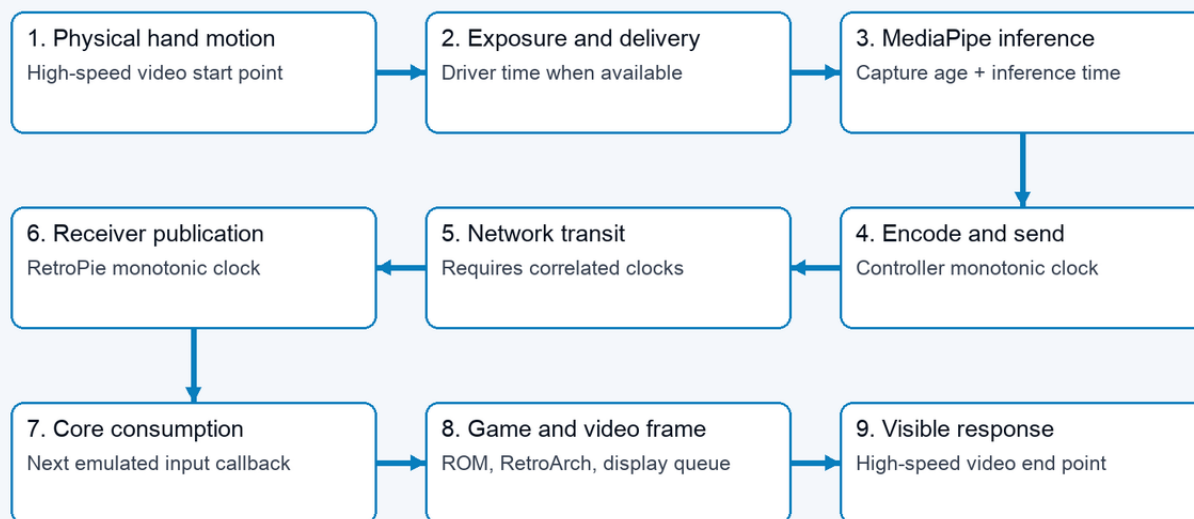
Repeat with `--phase movement` and a new output file while making deliberate short steps and returns. Use separate runs for preview open and closed.

The report records aggregate timings and observation quality, not images. Check fresh-sample count, request errors, detection/calibration count, skipped capture, and local send success before comparing percentiles.

`sent_sample_age_ms` proves only that the Controller sent locally; it does not prove receiver acceptance. An idle window exits with code 2 and must not be described as zero latency.

Keep the stage boundaries separate

09 / Where end-to-end time is measured



One hand-and-screen recording measures the whole path. Software traces explain only their own clock domains.

End-to-end latency boundaries from the physical hand and camera through MediaPipe, the console, the emulator, and the visible game

- Camera timestamps begin after exposure unless the driver documents otherwise.
- Controller timestamps use the Controller's monotonic clock.
- Receiver and core timestamps use the console's monotonic clock.
- Video frames use the recording's presentation timeline.

Do not subtract independent clocks or add stage percentiles. Correlate events by session, sequence, guard, and publication timestamp, then describe unmeasured boundaries honestly.

Native latency and stationary-jitter session

Use this session when a release or hardware change needs complete physical evidence rather than a quick status comparison.

1. Prepare the scene

- Use the intended player, saved calibration, camera, console, core, and ROM.
- Close the Dashboard preview for the primary baseline.
- Place a high-frame-rate camera behind and slightly beside the player so the physical hand and screen are both visible and neither obscures the other.
- Make a short framing clip, verify its actual cadence, and keep lighting and game state unchanged across comparison runs.

The read-only preflight captures software identities, hashes, service state, selected non-secret settings, and temperatures. It records no image, landmark, token, address, or player name.

```
SH
python3 scripts/prepare-end-to-end-session.py \
  --controller-status 'http://CONTROLLER:8088/status?statistics=1' \
  --controller-ssh arduino@CONTROLLER --controller-identity /path/to/controller-key \
  --retropie-ssh pi@CONSOLE --retropie-identity /path/to/console-key \
  --output-dir /tmp/virtualglove-preflight-01 --phase prepare
```

Run it again with a new output directory and `--phase record` after Super Glove Ball is running.

2. Run guided movement windows

```
SH
python3 scripts/run-native-latency-session.py \
  --status-url http://CONTROLLER.local:8088/status \
  --output-dir /tmp/virtualglove-session-01 --protocol full \
  --preflight /tmp/virtualglove-preflight-01/preflight.json
```

The full protocol guides three 20-second open-hand holds, then short and long moves in all four directions with holds and returns. `--protocol smoke` is a short framing check, not a performance baseline. The terminal cues pace the operator; they do not synchronize device clocks or identify physical onset.

3. Add bounded traces only when attribution is needed

Tracing is off by default. `scripts/manage-latency-traces.py` can temporarily enable bounded Controller and receiver traces, restart only their normal services, collect finalized files, and restore the normal environment.

```
SH
python3 scripts/manage-latency-traces.py start \
  --controller-ssh arduino@CONTROLLER --controller-identity /path/to/controller-key \
  --retropie-ssh pi@CONSOLE --retropie-identity /path/to/console-key \
  --duration 180 --state /tmp/virtualglove-trace-state.json

python3 scripts/manage-latency-traces.py stop \
  --state /tmp/virtualglove-trace-state.json \
  --output-dir /tmp/virtualglove-traces-01
```

Always run `stop`, including after a cancelled recording. Core consumption needs the separately named diagnostic Nestopia build and a normal game exit so its bounded CSV can finalize. Never replace the production core with that build.

Analyse matching files from one session:

```
SH
python3 scripts/analyze-latency-trace.py \
  --controller /tmp/virtualglove-controller.json \
  --receiver /tmp/virtualglove-receiver.json \
  --core /tmp/virtualglove-core.csv \
  --same-cabinet-boot --output /tmp/virtualglove-stages.json
```

Missing joins can mean window boundaries, overwritten state, or dropped trace evidence; they are not automatically packet loss. Review each trace's [dropped](#) and [stop_reason](#) fields.

4. Review the original video

Use [scripts/analyze-latency-video.py](#) on a preserved original recording. First create a contact sheet, then inspect likely onset frames and their immediate neighbours. Mark first physical hand motion and first matching Robo-Glove motion; do not use the terminal cue as onset.

Only accept measurements from a verified uniform-rate portion of the original. At 120 fps each frame spans about 8.3 ms; at 240 fps, about 4.2 ms. Report the frame bracket and annotation uncertainty alongside median, p95, and range. Stopping and stationary stability need their own annotations.

Private videos, traces, landmarks, and individual coordinates stay local. The project retains aggregate conclusions and selected non-sensitive evidence only.

Run the emulator benchmark again

Build the pinned cores for the host architecture in fresh directories, then run the deterministic comparison with legally supplied ROM paths:

```
SH
scripts/build-nestopia-powerglove.sh build/nestopia-response
scripts/build-fceumm-benchmark.sh build/fceumm-response

python3 scripts/benchmark-direction-response.py \
  --nestopia-core build/nestopia-response/nestopia_powerglove_libretro.so \
  --fceumm-core build/fceumm-response/fceumm_libretro.so \
  --super-glove-ball-rom '/path/to/Super Glove Ball (USA).nes' \
  --scratch /tmp/virtualglove-direction-scratch \
  --output /tmp/virtualglove-direction-response.json
```

Use the platform's actual library extension where appropriate. The runner checks ROM digests and core identities; do not reinterpret a run with mismatched inputs as comparable evidence.

The engineering lesson

The best native movement path was not the most complicated one. It emerged by repeatedly removing ambiguity:

- compare the same ROM from the same saved state;
- make the emulator reveal whether it consumed native or joypad input;
- keep only the newest camera frame;
- separate preview work from control delivery;
- replay identical pixels when comparing recognition settings;
- inspect cue-labelled misses instead of trusting a single percentage;
- use live play to reject optimizations that win only on paper;
- state exactly which boundaries remain unmeasured.

The finished system is faster because it does less speculative work. It follows the newest proven hand position, protects recovery without inventing motion, and releases safely when evidence becomes stale. That is what the validation was designed to establish.